

✦ JOSA //

Git Internals & Modern Workflow

/* Foundations refreshed and elevated */

OpenLab Apprenticeship 2026

Week 01

// 1 //

Learning Goals

- Understand what Git actually stores (objects, refs, the index) – not just the surface commands.
- Internalize the differences between the three main team workflows: GitHub Flow, Git Flow, and Trunk-Based Development.
- **Adopt Conventional Commits** and understand how it powers automated changelogs and SemVer.
- Recover gracefully from common Git mistakes without panicking or losing work.

// 2 //

Technical Deep Dive

// 2.1 What Git actually stores?

Git is, at its core, a content-addressable filesystem. Everything in a Git repository – every file you’ve ever committed, every commit, every tag, every branch – is one of just four object types stored under `.git/objects/`:

- **blob** – the contents of a file (no filename, no permissions, just bytes).
- **tree** – a directory listing: a list of (mode, name, blob-or-tree-hash) entries.
- **commit** – a pointer to a tree, plus parent commit(s), author, committer, message.
- **tag** – an annotated tag pointing to a commit (or any other object).

Each object is identified by the SHA-1 (now transitioning to SHA-256) hash of its contents. This is why Git is “immutable”: you can’t change history without changing every hash that follows. Try this in any repo:

```
# See the hash of HEAD
git rev-parse HEAD
```

```
# Look at the actual stored object
git cat-file -p HEAD
```

```
# Walk into the tree
git cat-file -p HEAD^{tree}
```

The output of the second command shows the commit's tree hash, parent hash, author, and message. The third shows the directory listing as Git stores it. Once you've seen this, commands like [rebase](#), [cherry-pick](#), and [reset](#) stop being magic — they are mechanical operations on this object graph.

A **branch** is just a file under `.git/refs/heads/` containing a commit hash. [HEAD](#) is a file containing either a branch name (attached) or a hash (detached). That's it.

// 2.2 Three workflows, one decision

- **GitHub Flow** — one long-lived branch (main), short-lived feature branches, PRs merge straight to main, deploy from main. Best for web apps and SaaS. This is what most modern teams use.
- **Git Flow** — *main* branch for releases, *develop* branch for integration, plus *feature/**, *release/**, and *hotfix/** branches. Designed for software with long release cycles. Now generally considered overkill for most projects.
- **Trunk-Based Development** — everyone commits directly (or via very short-lived branches) to a single trunk, behind feature flags, with continuous integration as the safety net. Used by Google, Facebook, and other high-velocity orgs.



TIP

Pragmatic default for OSS: GitHub Flow. Use it unless a project explicitly does something else.

// 2.3 Conventional Commits and why they matter

A Conventional Commit looks like this:

```
feat(parser): support trailing commas in JSON5 input
```

The previous lexer rejected `{"a": 1,}`. This PR loosens the grammar to accept it, matching the JSON5 spec.

BREAKING CHANGE: ``parse()`` now returns a richer error object.
Closes #482

The structure — `type(scope): subject` — looks trivial but unlocks tooling: **semantic-release** and **release-please** can read your commit log and automatically bump the version (`feat` → minor, `fix` → patch, `BREAKING CHANGE` → major), generate a CHANGELOG, and tag a release. No human writes the changelog.

This pairs naturally with SemVer: given [MAJOR.MINOR.PATCH](#), you bump MAJOR for breaking changes, MINOR for backwards-compatible features, and PATCH for backwards-compatible bug fixes.

Common types: [feat](#), [fix](#), [docs](#), [chore](#), [refactor](#), [test](#), [perf](#), [build](#), [ci](#), [style](#), [revert](#).

// 2.4 When Git scares you

Save yourself: bookmark ohshitgit.com. It's a one-page cheat sheet for the eight most common "I broke everything" Git situations and the exact commands to recover. The single most important rule: **if you committed it, it's almost never gone**. [git reflog](#) shows every position HEAD has been at for the last ~90 days.

```
git reflog          # find the SHA of "before I broke things"o
git reset --hard <sha> # back to that state
```

// 3 //////////////////////////////////

Learning Material

Topic	Resource
Git internals	Pro Git, Chapter 10: Git Internals
Building Git from scratch	Git from the Bottom Up – John Wiegley
Recovering from mistakes	Oh Shit, Git!?!
Workflow comparison	Atlassian: Comparing Workflows
Trunk-based development	trunkbaseddevelopment.com by Paul Hammant
Commit conventions	Conventional Commits 1.0.0 spec
Versioning	Semantic Versioning 2.0.0
Visual learning	Learn Git Branching (interactive)
GitHub workflow and first PR	GitHub Skills • “Introduction to GitHub” • “First week on GitHub” First Contribution Tutorial

// 4 //

Your Task for the Week

1. **Git archaeology.** Pick one of your previous commits in any of your repos. Use `git cat-file` to walk from the commit object to its tree to a blob. Write a one-paragraph reflection on what you saw and submit it as a Markdown in a repo.
2. **Reflog rescue drill.** Deliberately break a branch (e.g., `git reset --hard HEAD~5`), then recover the original state using `git reflog`. Take a screenshot of the recovery and add it to your the same Markdown file above.
3. **Refactor a commit history.** Take a feature branch with at least 3 messy commits and use `git rebase -i` to (a) rewrite all messages to follow Conventional Commits, (b) squash fixup commits, and (c) reorder if needed. Push it as a clean PR somewhere – even on a personal repo.

// 5 //

Soft Skill of the Week: Written Communication & Async Clarity

Open source is overwhelmingly **asynchronous**. The maintainer of the project you contributed to may live ten time zones away, may read your PR in a 90-second window between meetings, and may not have the context you have. Your writing is your interface.

The discipline you want to build is **bottom-line up front (BLUF)**: lead every message – whether it's a PR description, an issue comment, a Slack message, or an email – with the conclusion or the ask, then provide context. Compare:



HEADS UP

Don't: "I was working on the parser yesterday and noticed that when you have a deeply nested JSON object with arrays inside arrays, sometimes the indentation looks weird. I thought it might be related to the recent refactor. So I dug into it and..."



TIP

Do: "The pretty-printer mis-indents arrays nested 3+ levels deep. Repro and a fix in the linked PR. Root cause: the recursive depth counter wasn't being passed into the array branch."

// 5.1 Three rules to internalize

1. **The subject line / first sentence is a contract.** It tells the reader what the message is about and what you want from them.
2. **Show your work, don't just state your conclusion.** "Fixed it" is unreviewable. "I reproduced this on Node 18.17 with the attached test case; the fix is in parser.ts:142" is reviewable.
3. **Explicit beats clever.** If you want a review, ask for it. If you want a decision, name the question. Maintainers reading 50 notifications a day cannot afford to guess what you want.

// 5.2 Resources

- [GitLab Handbook – Asynchronous Communication](#)
- [Patrick McKenzie – Don't Call Yourself A Programmer](#) (the section on writing skills as career leverage)